Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №1
з дисципліни «Методи та технології паралельного програмування»

Тема: «Засоби створення та керування потоками в паралельних мультипоточних програмах»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Лесик Валентин Олександрович

Дата здачі: 12.03.2026

КИЇВ – 2026

Мета роботи. Ознайомитися з основними засобами створення та керування потоками в паралельних мультипоточних програмах, а також дослідити ефективність паралельної обробки для різних типів задач.

Завдання. Реалізувати **послідовну** та **паралельну мультипоточну** обробку незалежних задач.

Треба реалізувати приклади задач трьох типів:

1. **CPU-bound** – складні обчислення з невеликим обсягом даних.
2. **Memory-bound** – інтенсивна робота з даними в оперативній пам'яті.
3. **I/O-bound** – робота з даними на диску.

Відповідно, треба реалізувати 5 задач:

1. **(CPU-Bound)** Обрахунок числа π методом Монте-Карло з різною кількістю ітерацій;
2. **(CPU-Bound)** Факторизація великих чисел;
3. **(CPU-Bound)** Обчислення простих чисел в заданому діапазоні;
4. **(Memory-Bound)** Транспонувати матрицю великої розмірності `(10000 x 10000)`;
5. **(I/O-Bound)** Рекурсивно відкрити директорію та підрахувати загальну кількість слів у всіх текстових файлах; текстові файли та директорію згенерувати випадковим чином, додати значну кількість файлів – до 1000, в допустимих межах.

Виміряти та проаналізувати *залежність часу виконання від кількості потоків* для кожного типу задач.

Результати роботи повинні містити виконання логіки програми як і без застосування паралелізації, так і зі застосуванням за різних гіперпараметрів – кількості потоків, процесів, worker-ів; для кожного рішення треба знайти час виконання та зробити висновки, що впливає на прискорення та які треба підбирати гіперпараметри.

У звіт треба включити короткий опис обраного підходу до рішення задачі, опис критичних моментів рішення, код проекту, обґрунтування обрання фреймворку, опис відмінностей та переваг, рисунки із демонстрацією виконання програми та рисунки із демонстрацією прискорення та різними значеннями параметрів.

У відповідях на контрольні запитання слід вказувати на особливості та відмінності обраного студентом фреймворку, так як в залежності від обраної мови буде різнитися реалізація, підходи та можливості застосування.

Хід роботи.

Обґрунтування обрання інструментів та фреймворків.

Основною мовою програмування для реалізації бенчмарків було обрано Rust. Ця мова забезпечує високу продуктивність, порівнянну з C та C++, але при цьому гарантує безпеку роботи з пам'яттю на етапі компіляції завдяки концепціям володіння (`ownership`) та позичання (`borrowing`). Відсутність збирача сміття (`garbage collector`) дозволяє отримувати максимально точні та передбачувані результати під час вимірювання часу виконання, що є критично важливим для CPU-bound та Memory-bound задач. Крім того, Rust пропонує парадигму безпечного паралелізму, яка унеможливлює стани гонитви (data races) на рівні компіляції.

Для реалізації багатопотоковості на базовому рівні використовувався стандартний модуль `std::thread`. Цей інструмент надає прямий доступ до потоків операційної системи. Використання областей видимості потоків (`thread::scope`) дозволило безпечно розділяти посилання на спільні дані без необхідності застосування додаткових примітивів синхронізації, таких як `Arc`. Це дало змогу оцінити «чистий» час виконання алгоритмів при жорсткому ручному розподілі навантаження.

У якості пулу воркерів (`worker pool`) було інтегровано бібліотеку `rayon`, яка є стандартом де-факто для паралелізму даних у екосистемі Rust. Головною перевагою цього фреймворку є алгоритм динамічного балансування навантаження (`work-stealing`). Замість статичного поділу даних між потоками, `rayon` дозволяє воркерам автоматично забирати частину роботи у більш завантажених сусідів після завершення власних завдань. Це архітектурне рішення максимізує утилізацію процесорних ядер, особливо у задачах з нерівномірною обчислювальною складністю.

Для аналізу багатопроцесового режиму було застосовано підхід із використанням легковагового міжпроцесового зв'язку (`Lightweight IPC`). Замість залучення важких сторонніх фреймворків, головний оркестратор створює повністю ізольовані дочірні процеси через виклики командного рядка та агрегує їхні результати через стандартний вивід (`stdout`). Такий підхід наочно демонструє накладні витрати операційної системи на створення нових процесів та ізольоване виділення сторінок пам'яті, що стало особливо помітним під час тестування задач, інтенсивних до роботи з оперативною пам'яттю.

Завдання 1. *Обрахунок числа π методом Монте-Карло з різною кількістю ітерацій.*

Метод Монте-Карло базується на законі великих чисел та геометричній ймовірності. Суть алгоритму полягає у випадковому генеруванні точок з координатами (x, y) у межах квадрата зі стороною 2 (від -1.0 до 1.0). У цей квадрат вписано коло радіусом 1. Якщо сума квадратів координат згенерованої точки менша або дорівнює одиниці ($x^2 + y^2 \leq 1.0$), точка вважається такою, що потрапила всередину кола.

Відношення площі кола до площі квадрата дорівнює $\frac{\pi}{4}$. Відповідно, помноживши відношення кількості точок, що потрапили в коло, до загальної кількості згенерованих точок на 4, ми отримуємо наближене значення числа π. Математично це виражається формулою:

$$\pi \approx 4 \times \frac{N_{hits}}{N_{total}}$$

Чим більша кількість ітерацій (згенеруваних точок), тим вищою є точність розрахунку. Це типова процесорно-інтенсивна (CPU-bound) задача, оскільки вона вимагає постійної роботи генератора псевдовипадкових чисел та виконання базових арифметичних операцій.

Критичні моменти рішення. Першим критичним моментом є ізоляція генератора псевдовипадкових чисел (RNG). Використання єдиного глобального генератора для всіх потоків призвело б до стану гонитви або величезних затримок через блокування (м'ютекси). Тому кожен потік або процес ініціалізує власний локальний генератор `rand::rng()` у межах своєї порції обчислень.

Другим важливим аспектом є уникнення спільного доступу до пам'яті під час ітерацій. Замість інкрементування спільного атомарного лічильника на кожній ітерації, кожен обчислювальний юніт підраховує локальну кількість влучань (`hits`). Агрегація результатів відбувається лише один раз наприкінці роботи потоку, що максимізує пропускну здатність ядер процесора.

Третім моментом є цілочисельне ділення кількості ітерацій між процесами. Щоб уникнути похибок через залишок від ділення, фінальний результат розраховується на основі фактично виконаної кількості ітерацій (`real_iterations`), а не початково заданої.

Аналіз результатів та підбір гіперпараметрів. Як видно на скріншоті нижче, всі методи паралелізації успішно впоралися із завданням, повернувши коректне наближене значення числа π (близько 3.141...). Це підтверджує математичну

правильність розподілу ітерацій та відсутність втрати даних під час агрегації.

```
monte_carlo.rs    start.sh    2026-12-03.log
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: Sequential, RESULT: 3.14166992
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 2 threads, RESULT: 3.14173828
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 4 threads, RESULT: 3.14170756
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 8 threads, RESULT: 3.14142852
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 2 workers, RESULT: 3.14165448
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 4 workers, RESULT: 3.14180476
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 8 workers, RESULT: 3.14153076
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 2 processes, RESULT: 3.14144416
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 4 processes, RESULT: 3.1416522
[2026-03-12 01:33 INFO] TASK: CPU-Bound: Monte-Carlo, MODE: 8 processes, RESULT: 3.14178328
```

Аналіз часу виконання демонструє класичну поведінку CPU-bound задачі. Послідовне виконання на одному ядрі зайняло 0.7062 секунди. Застосування паралелізму дало майже лінійне прискорення. Найкращий результат показав пул воркерів бібліотеки Rayon на 8 ядрах – 0.1369 секунди, що більш ніж у 5 разів швидше за базовий варіант. Окремі потоки та процеси показали дещо гірший, але близький результат (0.1735 та 0.1707 секунди відповідно на 8 юнітах). Невелике відставання звичайних потоків пояснюється накладними витратами на їх створення та очікування завершення через `join`, тоді як `rayon` ефективно перерозподіляє завдання всередині вже існуючого пулу.

```
xairaven@laptop ~/Files/Programming/KPI-MTPP <main>
$ ./Lab1/start.sh
    Finished `release` profile [optimized] target(s) in 0.07s
=== REPORT FOR CPU-BOUND: MONTE-CARLO ===
- Sequential -
Sequential: 0.7062 seconds

- Multi-thread -
2 threads: 0.3806 seconds
4 threads: 0.2775 seconds
8 threads: 0.1735 seconds

- Worker pool -
2 workers: 0.3228 seconds
4 workers: 0.2309 seconds
8 workers: 0.1369 seconds

- Multi-process -
2 processes: 0.3869 seconds
4 processes: 0.2895 seconds
8 processes: 0.1707 seconds
```

Головним гіперпараметром для оптимізації цього типу задач є кількість потоків або воркерів. Для досягнення максимального прискорення це значення повинно дорівнювати кількості фізичних або логічних ядер процесора. Подальше збільшення гіперпараметра не дасть приросту швидкості, а навпаки, призведе до уповільнення через надмірне перемикання контексту операційною системою.

Завдання 2. *Факторизація великих чисел.*

Опис підходу до вирішення. Задача факторизації полягає у знаходженні всіх простих множників заданого складеного числа. Враховуючи обчислювальну складність перевірки великих чисел, ця задача є класичним прикладом навантаження на процесор (CPU-bound).

Для тестування було обрано метод пробного ділення, де цільове число перевіряється на подільність без остачі на послідовність натуральних чисел. Математичною межею для перевірки є квадратний корінь із цільового числа ($\sqrt{N}$). Якщо до цієї межі не знайдено жодного дільника (окрім 1), число є простим, і подальший пошук не має сенсу. Щоб максимально утилізувати процесор під час бенчмарку, у якості цільового числа було обрано напівпросте число 18446743979220271189, яке є добутком двох великих простих чисел (4294967291 та 4294967279). Це змушує алгоритм виконувати ітерації майже до самої межі квадратного кореня (понад 4 мільярди ітерацій).

Для оптимізації базового алгоритму застосовано математичне відсікання парних чисел. Після перевірки на подільність на 2 алгоритм примусово робить стартову позицію непарною і далі крокує лише по непарних числах (`current += 2`). Це зменшує загальну кількість необхідних операцій ділення для процесора рівно вдвічі.

Критичні моменти рішення. Ключовим викликом при розпаралелюванні факторизації є правильний математичний розподіл простору пошуку між потоками. Просте цілочисельне ділення верхньої межі на кількість потоків призвело б до втрати «хвоста» діапазону. Тому для розрахунку розміру порції застосовується ділення з округленням вгору (`div_ceil`). Стартова та кінцева позиції для кожного потоку жорстко обмежуються за допомогою функцій `max` та `min`, щоб запобігти діленню на нуль або виходу за межі квадратного кореня.

Другим критичним моментом є обробка стану гонитви (race condition). Оскільки різні потоки або процеси шукають дільники у своїх незалежних діапазонах, вони можуть знайти різні множники майже одночасно. У потоках ця проблема вирішується використанням `AtomicU64` та методу `compare_exchange`, що дозволяє безпечно записати перший знайдений результат. Для процесів, де пам'ять ізольована, агрегатор збирає всі результати від дочірніх процесів та обирає найменше валідне значення (мінімум серед знайдених дільників), гарантуючи коректність кінцевої відповіді незалежно від порядку завершення процесів.

Аналіз результатів та підбір гіперпараметрів. Як видно на скріншоті логів нижче, математична логіка, розподіл діапазонів та агрегація працюють без-

доганно: всі без винятку режими паралелізму повернули правильний найменший дільник – 4294967279.

```
start.sh    2026-12-03.log    task.rs
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Sequential, RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: 2 threads, RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: 4 threads, RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: 8 threads, RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Workers(2), RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Workers(4), RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Workers(8), RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Processes(2), RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Processes(4), RESULT: 4294967279
[2026-03-12 01:42 INFO] TASK: CPU-Bound: Factorization, MODE: Processes(8), RESULT: 4294967279
```

Аналіз часу виконання підтверджує очікуваний профіль CPU-bound задачі з ідеальним масштабуванням. Послідовне виконання на одному ядрі зайняло 5.1001 секунди. Зі збільшенням кількості потоків та процесів час виконання пропорційно зменшувався. Так, перехід від 2 до 8 потоків скоротив час з 2.82 до 1.14 секунди.

```
xairaven@laptop ~/Files/Programming/KPI-MTPP <main●>
$ ./Lab1/start.sh
    Finished `release` profile [optimized] target(s) in 0.02s
=== REPORT FOR CPU-BOUND: FACTORIZATION ===
- Sequential -
Sequential: 5.1001 seconds

- Multi-thread -
2 threads: 2.8204 seconds
4 threads: 2.0248 seconds
8 threads: 1.1485 seconds

- Worker pool -
2 workers: 2.7837 seconds
4 workers: 1.7403 seconds
8 workers: 0.9938 seconds

- Multi-process -
2 processes: 2.7058 seconds
4 processes: 1.9310 seconds
8 processes: 1.1143 seconds
```

Як і в попередньому завданні, пул воркерів Rayon продемонстрував найвищу ефективність на 8 ядрах (0.9938 секунди). Це пояснюється тим, що `rayon` розбиває загальний діапазон на набагато більшу кількість дрібних чанків (у даному випадку `workers * 10`) та використовує динамічний work-stealing, компенсуючи будь-які мікрозатримки окремих ядер.

Оптимальним гіперпараметром для цієї задачі є кількість потоків, що відповідає кількості фізичних ядер процесора. Збільшення розміру пулу чанків (наприклад, `workers * 100`) у Rayon могло б ще трохи згладити балансування навантаження, але для ручних потоків та процесів кількість юнітів має суворо дорівнювати доступним апаратним ресурсам, щоб уникнути витрат на перемикання контексту операційною системою.

Завдання 3. *Обчислення простих чисел в заданому діапазоні.*

Опис підходу до вирішення. Задача знаходження всіх простих чисел у великому діапазоні (у даному випадку від 1 до 10000000) є типовою задачею, що інтенсивно використовує ресурси процесора (CPU-bound). Оскільки використання масивів зі спільним доступом (як у Решеті Ератосфена) призвело б до перетворення задачі на Memory-bound, було обрано підхід незалежної перевірки кожного числа методом пробного ділення.

Для оптимізації математичних перевірок було застосовано алгоритм на основі концепції $6k \pm 1$. Згідно з цією теорією, всі прості числа, більші за 3, знаходяться поруч із числами, кратними шести. Алгоритм швидко відкидає числа, що діляться на 2 та 3, а потім крокує циклом з інтервалом 6, перевіряючи лише пари i та $i + 2$ аж до квадратного кореня цільового числа. Це значно зменшує кількість необхідних математичних операцій для кожного окремого числа.

Критичні моменти рішення. Основною проблемою в цій задачі є нерівномірний розподіл навантаження. Перевірка великих чисел (ближче до 10 мільйонів) вимагає значно більше процесорних циклів, ніж перевірка малих чисел. Якщо просто поділити загальний масив чисел на безперервні блоки (наприклад, по 1 мільйону чисел на потік), потоки з великими числами будуть працювати значно довше за інші, що призведе до «ефекту пляшкового горла» та простою більшості ядер процесора.

Для вирішення цієї проблеми у багатопотоковому та багатопроцесовому режимах було застосовано циклічний розподіл даних (interleaved distribution). Кожен потік не бере суцільний блок, а крокує через увесь діапазон з інтервалом, що дорівнює загальній кількості потоків (`current += threads`). Це гарантує ідеальне математичне балансування, оскільки кожен обчислювальний юніт отримує однакову суміш малих, середніх та великих чисел.

У пулі воркерів `rayon` цю проблему елегантно вирішено завдяки використанню паралельних ітераторів (`into_par_iter`). Rayon автоматично розбиває ітератор на дрібні чанки і використовує алгоритм work-stealing для динамічного перерозподілу завдань між воркерами «на льоту».

Аналіз результатів та підбір гіперпараметрів. Як видно на скріншоті нижче, всі методи паралелізації знайшли ідентичну кількість простих чисел у заданому діапазоні – 664579. Це підтверджує коректність математичного чергування у потоках і процесах, а також правильність агрегації результатів.

```
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Sequential, RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: 2 threads, RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: 4 threads, RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: 8 threads, RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Workers(2), RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Workers(4), RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Workers(8), RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Processes(2), RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Processes(4), RESULT: 664579
[2026-03-12 01:44 INFO] TASK: CPU-Bound: Primes, MODE: Processes(8), RESULT: 664579
```

Аналіз часу виконання виявляє цікаву особливість циклічного розподілу для 2 потоків. Послідовне виконання зайняло 0.8847 секунди, а режим із двома потоками/процесами – 0.8582 / 0.8952 секунди. Фактично прискорення відсутнє. Це сталося через специфіку алгоритму перевірки: один потік отримував виключно парні числа, миттєво відкидав їх через умову `is_multiple_of(2)` і завершував роботу, залишаючи всю «важку» математику для іншого потоку.

```
=== REPORT FOR CPU-BOUND: PRIMES ===
- Sequential -
Sequential: 0.8847 seconds

- Multi-thread -
2 threads: 0.8582 seconds
4 threads: 0.4562 seconds
8 threads: 0.3180 seconds

- Worker pool -
2 workers: 0.4881 seconds
4 workers: 0.2937 seconds
8 workers: 0.1711 seconds

- Multi-process -
2 processes: 0.8952 seconds
4 processes: 0.4666 seconds
8 processes: 0.3257 seconds
```

Проте, зі збільшенням кількості потоків та процесів до 4 і 8 баланс відновлюється, і ми спостерігаємо хороше масштабування (до 0.318 секунд на 8 потоках).

Абсолютним лідером знову став пул воркерів `rayon`. На 8 воркерах час виконання склав лише 0.1711 секунди (приблизно в 5 разів швидше за послідовний код). Динамічне балансування Rayon виявилося значно ефективнішим за ручне математичне чергування, оскільки воно не залежить від специфічних патернів у вхідних даних (як-от чергування парних та непарних чисел).

Для задач такого типу головним гіперпараметром є архітектурний підхід до балансування. Якщо використовується `rayon`, гіперпараметр (кількість воркерів) слід прирівнювати до кількості ядер процесора. Якщо використовуються ручні потоки, необхідно ретельно аналізувати специфіку даних перед вибором алгоритму розподілу чанків, щоб уникнути помилкової «ілюзії розпаралелювання».

Завдання 4. *Транспонувати матрицю великої розмірності* `(10000 x 10000)`.

Опис підходу до вирішення. Задача транспонування матриці (заміна рядків на стовпці за правилом $A^T_{i,j} = A_{j,i}$) кардинально відрізняється від попередніх завдань своєю природою. Тут практично відсутнє математичне навантаження на процесор, натомість головним «вузьким місцем» стає пропускна здатність оперативної пам'яті (Memory-bound). Для матриці розміром 10000×10000 елементів типу `u64` потрібно перемістити близько 800 мегабайтів даних.

Головна проблема полягає в архітектурі процесорних кешів. При читанні масиву по рядках кеш працює ефективно, завантажуючи цілі блоки даних (кеш-лінії). Однак при транспонуванні запис або читання неминуче відбувається по стовпцях. Елементи одного стовпця знаходяться далеко один від одного у фізичному адресному просторі пам'яті. Це спричиняє масові промахи кешу (cache misses), через що процесор змушений постійно простоювати, очікуючи на повільну вибірку даних з оперативної пам'яті.

Критичні моменти рішення. Критичним викликом при розпаралелюванні запису в пам'ять є уникнення станів гонитви (data races) та доведення компілятору Rust безпеки таких операцій. Ця проблема елегантно вирішується за допомогою методу `chunks_mut`, який розрізає вихідний масив на гарантовано незалежні, неперетинні мутабельні фрагменти (слайси). Кожен потік отримує ексклюзивне право на запис у свій діапазон рядків, що робить алгоритм абсолютно безпечним.

Другим критичним моментом стала ізоляція пам'яті в багатопроцесовому режимі. Оскільки дочірні процеси не успадковують стан оперативної пам'яті оркестратора, масив вхідних даних у них виявлявся порожнім. Щоб уникнути помилок доступу за межі масиву (`IndexingOutOfBounds`), алгоритм використовує детерміновану генерацію вхідних значень. Якщо процес фіксує порожній вхідний масив, він локально виділяє необхідні 800 МБ пам'яті та заповнює їх за тією ж

формулою, що й головний процес.

Також варто відзначити метод перевірки коректності (sanity check). Перевірка всієї матриці після бенчмарку вимагала б ще 100 мільйонів операцій читання, що суттєво спотворило б результати тестування. Тому алгоритм виконує $O(1)$ перевірку лише кількох асиметричних елементів (наприклад, кутових), що гарантує цілісність логіки без втрати часу.

Аналіз результатів та підбір гіперпараметрів. Як видно на скріншоті нижче, всі паралельні підходи успішно пройшли швидку перевірку цілісності (`IS TRANSPOSED: true`), а агрегатор процесів коректно зібрав контрольне значення, що підтверджує правильність обчислень.

```
start.sh    2026-12-03.log ×    task.rs    transpose.rs
1   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: Sequential, IS TRANSPOSED: true
2   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 2 threads, IS TRANSPOSED: true
3   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 4 threads, IS TRANSPOSED: true
4   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 8 threads, IS TRANSPOSED: true
5   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 2 workers, IS TRANSPOSED: true
6   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 4 workers, IS TRANSPOSED: true
7   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: 8 workers, IS TRANSPOSED: true
8   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: Processes(2), RESULT: 0
9   [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: Processes(4), RESULT: 0
10  [2026-03-12 02:34 INFO] TASK: Memory-Bound: Matrix Transpose, MODE: Processes(8), RESULT: 250
```

Проте аналіз часу виконання яскраво ілюструє феномен Memory-Bound обмежень. Послідовне виконання зайняло 1.0265 секунди. Перехід до багатопотоковості (Threads) та пулу воркерів (`rayon`) дав певне прискорення, зменшивши час до приблизно 0.41 - 0.48 секунди. Однак класичного лінійного масштабування не відбулося: 8 потоків або воркерів працюють майже з такою ж швидкістю, як і 2. Це доводить, що пропускна здатність шини оперативної пам'яті (Memory Bandwidth) була повністю вичерпана вже на перших двох потоках. Додавання нових обчислювальних ядер не прискорює програму, оскільки контролер пам'яті фізично не здатний передавати дані швидше.

```
=== REPORT FOR MEMORY-BOUND: MATRIX TRANSPOSE ===
- Sequential -
Sequential: 1.0265 seconds

- Multi-thread -
2 threads: 0.4823 seconds
4 threads: 0.6290 seconds
8 threads: 0.4101 seconds

- Worker pool -
2 workers: 0.4533 seconds
4 workers: 0.4253 seconds
8 workers: 0.4349 seconds

- Multi-process -
2 processes: 1.3173 seconds
4 processes: 1.4873 seconds
8 processes: 2.3979 seconds
```

Ще цікавішою є поведінка багатопроцесового режиму. Час виконання тут не просто не зменшився, а катастрофічно зріс: від 1.31 секунди на двох процесах до 2.39 секунди на вісьмох. Цей антипатерн масштабування пояснюється величезними накладними витратами операційної системи. Для кожного з 8 дочірніх процесів ОС змушена була виділяти окремі ізольовані 800 МБ пам'яті та обробляти тисячі помилок відсутності сторінок (page faults) під час ініціалізації `fallback_input`.

Підбір гіперпараметрів для Memory-Bound задач кардинально відрізняється від CPU-Bound. Оптимальним значенням кількості потоків є мінімально необхідне число для насичення шини пам'яті (зазвичай 2 або 4 потоки, залежно від архітектури каналів пам'яті материнської плати). Подальше збільшення цього гіперпараметра є марною тратою процесорного часу. Використання ізольованих процесів для задач із великим спільним станом пам'яті є вкрай неефективним і повинно уникатися.

Завдання 5. *Рекурсивно відкрити директорію та підрахувати загальну кількість слів у всіх текстових файлах; текстові файли та директорію згенерувати випадковим чином, додати значну кількість файлів – до 1000, в допустимих межах.*

Опис підходу до вирішення. Задача підрахунку загальної кількості слів у великій кількості файлів є класичним прикладом навантаження на підсистему вводу-виводу (I/O-bound). На відміну від процесора чи оперативної пам'яті, фізичні накопичувачі (навіть сучасні NVMe SSD) є найповільнішими компонентами системи. Під час синхронного читання файлу процесор змушений простоювати, очікуючи на відповідь від контролера диска.

Для тестування було реалізовано алгоритм, який спочатку створює тимчасову директорію зі складною ієрархією вкладених папок, у які генерується 1000 текстових файлів (по 100 слів у кожному). Основний етап бенчмарку полягає у рекурсивному обході цієї директорії, відкритті кожного знайденого файлу, зчитуванні його вмісту в рядок та підрахунку слів за допомогою поділу по пробільних символах.

Щоб зменшити час простою процесора, паралельні підходи (потоки, пули воркерів та процеси) одночасно надсилають операційній системі множинні запити на читання різних файлів. Це дозволяє контролеру диска оптимізувати чергу запитів та віддавати дані паралельно.

Критичні моменти рішення. Першим критичним моментом є гарантування детермінованості під час розподілу файлів між обчислювальними юнітами.

Оскільки функція читання директорій операційної системи не гарантує однакового порядку повернення файлів при різних викликах, алгоритм примусово сортує зібраний масив шляхів (`files.sort()`). Без цього кроку процеси могли б обробити одні й ті самі файли двічі або взагалі пропустити частину даних.

Другим важливим аспектом є обробка помилок вводу-виводу. Робота з файловою системою завжди пов'язана з ризиком відмови у доступі або блокування файлів іншими програмами. У коді застосовано безпечне зіставлення з шаблоном (`match`), яке у разі виникнення помилки логує проблему із зазначенням конкретного шляху та коректно перериває виконання потоку або процесу, запобігаючи прихованій втраті даних.

Третім, і найцікавішим, моментом є вплив механізмів кешування операційної системи, який необхідно враховувати при проектуванні будь-яких I/O бенчмарків.

Аналіз результатів та підбір гіперпараметрів. Як видно на скріншотах логів нижче, агрегація результатів працює коректно: всі режими паралелізму повернули ідентичний результат – 100 000 слів, що повністю відповідає згенерованим даним (1000 файлів по 100 слів).

```
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Sequential, RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: 2 threads, RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: 4 threads, RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: 8 threads, RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Workers(2), RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Workers(4), RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Workers(8), RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Processes(2), RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Processes(4), RESULT: 100000
[2026-03-12 02:45 INFO] TASK: I/O-Bound: Word Count, MODE: Processes(8), RESULT: 100000
```

Аналіз метрик часу виконання демонструє специфічну поведінку, зумовлену втручанням операційної системи. Послідовне виконання зайняло всього 0.0050 секунди. Найшвидший паралельний метод (8 воркерів Rayon) виконав завдання за 0.0019 секунди, а 8 ізольованих процесів – за 0.0042 секунди. Ці показники є аномально швидкими для операцій з фізичним диском.

Така швидкість пояснюється механізмом Page Cache. Під час ініціалізації масиву даних (фаза `setup`) операційна система записала згенеровані файли на диск, але водночас залишила їхні копії у швидкій оперативній пам'яті. Коли бенчмарк почав вимірювання часу та звернувся до цих файлів, операційна система не стала робити повільні запити до диска, а миттєво віддала дані з оперативної пам'яті. Таким чином, задача фактично перетворилася з I/O-bound на гібридну

CPU/Memory-bound задачу. Прискорення з 0.0050 до 0.0019 секунди на 8 воркерах відображає швидкість паралельного поділу прочитаних рядків на слова, а не швидкість читання з накопичувача. Збільшення часу в процесах (до 0.0042 секунди) знову ілюструє накладні витрати на їхній запуск.

```
=== REPORT FOR I/O-BOUND: WORD COUNT ===
- Sequential -
Sequential: 0.0050 seconds

- Multi-thread -
2 threads: 0.0041 seconds
4 threads: 0.0033 seconds
8 threads: 0.0028 seconds

- Worker pool -
2 workers: 0.0041 seconds
4 workers: 0.0028 seconds
8 workers: 0.0019 seconds

- Multi-process -
2 processes: 0.0051 seconds
4 processes: 0.0041 seconds
8 processes: 0.0042 seconds
```

Підбір гіперпараметрів для справжніх I/O-bound задач залежить від типу накопичувача. Для класичних жорстких дисків (HDD) надмірна кількість потоків може призвести до уповільнення через фізичне переміщення магнітної головки. Для сучасних NVMe накопичувачів кількість потоків може бути значно вищою і обмежується лише глибиною черги контролера та кількістю ядер процесора, здатних швидко обробляти отримані байти.

Висновок. У ході виконання лабораторної роботи було успішно реалізовано та проаналізовано паралельну обробку для трьох кардинально різних типів навантажень: CPU-bound, Memory-bound та I/O-bound. Дослідження довело, що вибір архітектури паралелізму та гіперпараметрів має суворо залежати від природи «вузького місця» системи.

Для CPU-bound задач (метод Монте-Карло, факторизація, пошук простих чисел) розпаралелювання продемонструвало майже лінійне прискорення. Найвищу ефективність показав пул воркерів бібліотеки Rayon завдяки своєму алгоритму динамічного балансування (work-stealing), який нівелює проблему нерівномірної складності обчислень. Максимальне прискорення тут обмежується виключно кількістю фізичних ядер процесора.

Для Memory-bound задач (транспонування матриці) результати виявилися контрінтуїтивними на перший погляд. Продуктивність впирається у пропускну здатність шини оперативної пам'яті (Memory Bandwidth), тому збільшення кількості потоків більше 2-4 не дає жодного приросту швидкості. Багатопроцесовий підхід у цьому сценарії взагалі виявився антипатерном через колосальні накладні витрати операційної системи на виділення ізольованих сторінок пам'яті.

Тестування I/O-bound задачі (підрахунок слів у файлах) яскраво продемонструвало вплив кешування операційної системи (Page Cache). Повторні звернення до файлів обробляються безпосередньо з оперативної пам'яті, що перетворює задачу на процесорно-залежну. Тим не менш, загальний принцип роботи з I/O доводить, що для таких задач кількість потоків може значно перевищувати кількість ядер процесора, оскільки потоки більшу частину часу очікують на відповідь від апаратного контролера.

Контрольні запитання.

1. **Які були застосовані фреймворки та які їх особливості та відмінності від інших фреймворків?**

   У роботі використовувався вбудований модуль `std::thread`, який забезпечує пряме відображення (1:1) на потоки операційної системи та дозволяє безпечно працювати зі спільними посиланнями через механізм `thread::scope`. На відміну від нього, сторонній фреймворк `rayon` використовує парадигму пулу воркерів та паралельних ітераторів. Його головна відмінність полягає в алгоритмі work-stealing, який автоматично перерозподіляє завдання між потоками «на льоту», звільняючи програміста від ручного вираховування розмірів чанків. Багатопроцесовий підхід реалізовувався через виклики CLI та Lightweight IPC, що забезпечує найвищий рівень ізоляції пам'яті, але вимагає ручної агрегації результатів та має найбільший час запуску порівняно з потоками.

2. **Поясніть різницю між CPU-bound, Memory-bound та I/O-bound задачами.**

   CPU-bound задачі обмежуються виключно швидкістю обчислювальних блоків (ALU) процесора, наприклад, при складних математичних розрахунках. Memory-bound задачі лімітуються швидкістю та пропускною здатністю оперативної пам'яті; у таких задачах процесор часто простоює в очікуванні даних через промахи кешу (cache misses). I/O-bound задачі є найбільш повільними, оскільки швидкість виконання залежить від зовнішніх пристроїв:

жорстких дисків, мережевих адаптерів або периферії, змушуючи процесор переходити у стан сну під час очікування апаратних переривань.

3. **Що таке закон Амдала і як він впливає на прискорення паралельних програм?**

   Закон Амдала визначає теоретичну межу максимального прискорення програми при використанні паралельних обчислень. Він стверджує, що загальне прискорення завжди обмежене тією часткою алгоритму, яка може бути виконана лише послідовно. Наприклад, якщо 10% коду програми принципово не піддається розпаралелюванню, то навіть при використанні нескінченної кількості процесорних ядер максимальне прискорення всієї програми ніколи не перевищить 10 разів. Це доводить, що бездумне додавання потоків дає спадаючу віддачу.

4. **Чому I/O-bound задачі можуть ефективно використовувати більше потоків, ніж кількість ядер процесора?**

   Під час виконання I/O-bound задач потік викликає системний запит до диска або мережі і переходить у заблокований стан (sleep state). Оскільки в цей час він не споживає ресурсів арифметико-логічного пристрою, планувальник операційної системи може миттєво передати ядро процесора іншому потоку. Таким чином, створення сотень потоків для читання файлів на 8-ядерному процесорі дозволяє тримати апаратну чергу накопичувача постійно заповненою, максимізуючи пропускну здатність вводу-виводу.

5. **Що таке overhead при створенні потоків і як він впливає на продуктивність?**

   Overhead (накладні витрати) – це час та ресурси пам’яті, які операційна система витрачає на системні виклики для виділення стеку нового потоку, його реєстрацію в планувальнику та подальше перемикання контексту (збереження та відновлення регістрів процесора). Якщо завдання, яке передається потоку, виконується за кілька мікросекунд, накладні витрати на створення та завершення потоку значно перевищать час самих корисних обчислень, що призведе до сильного уповільнення програми порівняно з послідовним виконанням.

6. **Поясніть концепцію пулу потоків (`thread pool`) та його переваги.**

   Пул потоків – це архітектурний патерн, при якому заздалегідь створюється фіксована кількість робочих потоків (зазвичай рівна кількості логічних ядер), які ніколи не завершуються, а постійно очікують на нові завдання із загальної черги. Основна перевага полягає у повному уникненні overhead’у

на постійне створення та знищення потоків. Крім того, пул потоків захищає систему від вичерпання ресурсів (out of memory), оскільки обмежує максимальну кількість одночасно активних потоків, навіть якщо завдань у черзі мільйони.

7. **Які проблеми можуть виникати при роботі з Memory-bound задачами в багатопоточному середовищі?**

   Основною проблемою є фізичне насичення шини пам’яті. Коли безліч ядер одночасно запитують великі обсяги різних даних, контролер пам’яті не встигає їх обслуговувати, створюючи апаратний затор. Додатково виникає проблема інвалідації кешів (false sharing): якщо два потоки одночасно змінюють різні змінні, що знаходяться в одній фізичній кеш-лінії процесора, підсистема пам’яті змушена постійно перезаписувати цю лінію між L1-кешами різних ядер, що катастрофічно знижує швидкість роботи всього алгоритму.

Лістинг.

monte_carlo.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkMetadata, Benchmarkable};
use crate::task::executor::Executable;
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use rand::RngExt;
use rayon::iter::IntoParallelIterator;
use rayon::iter::ParallelIterator;
use std::thread;

pub struct MonteCarlo {
    iterations: usize,
}

// Builder
impl MonteCarlo {
    pub fn with_iterations(iterations: usize) -> Self {
        Self { iterations }
    }
}

// Core logic
impl MonteCarlo {
    const RANGE: std::ops::RangeInclusive<f64> = -1.0..=1.0;

    // Core mathematical logic for generating random points and counting circle hits.
    // Isolated here to be easily reused across different execution modes.
    fn calculate_chunk(iterations: usize) -> usize {
        let mut rng = rand::rng();
        let mut hits = 0;

        for _ in 0..iterations {
            let x: f64 = rng.random_range(Self::RANGE);
            let y: f64 = rng.random_range(Self::RANGE);

            // Check if the generated point falls inside the unit circle
            if x * x + y * y <= 1.0 {
                hits += 1;
            }
        }

        hits
    }

    // Estimates Pi based on the total hits and total iterations.
    fn calculate_result(&self, total_hits: usize, real_iterations: usize) -> f64 {
        4.0 * (total_hits as f64) / (real_iterations as f64)
    }
}

impl Benchmarkable for MonteCarlo {}
impl Measurable for MonteCarlo {}
impl Manageable for MonteCarlo {}

impl Reportable for MonteCarlo {
    fn name(&self) -> &'static str {
        "CPU-Bound: Monte-Carlo"
```

```
    }
}

impl Executable for MonteCarlo {
    fn run_sequential(&self) -> Result<(), Error> {
        let hits = Self::calculate_chunk(self.iterations);
        let result = self.calculate_result(hits, self.iterations);

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Sequential,
        result
        );
        Ok(())
    }

    fn run_threads(&self, threads: usize) -> Result<(), Error> {
        let iterations_per_thread = self.iterations / threads;
        let mut handles = Vec::with_capacity(threads);

        for _ in 0..threads {
            handles.push(thread::spawn(move || {
                Self::calculate_chunk(iterations_per_thread)
            }));
        }

        let mut total_hits: usize = 0;
        for handle in handles {
            total_hits += handle.join().map_err(|_| SystemError::ThreadPanicked)?;
        }
        let result = self.calculate_result(total_hits, iterations_per_thread * threads);

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Threads(threads),
        result
        );
        Ok(())
    }

    fn run_workers(&self, workers: usize) -> Result<(), Error> {
        let iterations_per_worker = self.iterations / workers;

        // Use Rayon to create a parallel iterator over the number of workers
        let total_hits: usize = (0..workers)
        .into_par_iter()
        .map(|_| Self::calculate_chunk(iterations_per_worker))
        .sum();

        let result = self.calculate_result(total_hits, iterations_per_worker * workers);

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Workers(workers),
        result
        );
        Ok(())
    }

    fn run_process_chunk(
```

```rust
    &self, _process_index: usize, total_processes: usize,
    ) -> Result<(), Error> {
        // Each child process calculates its own chunk independently
        let iterations_per_process = self.iterations / total_processes;
        let hits = Self::calculate_chunk(iterations_per_process);

        println!("{}", hits);

        Ok(())
    }

    fn aggregate_process_results(
    &self, total_processes: usize, results: Vec<String>,
    ) -> Result<(), Error> {
        let mut total_hits = 0;

        // Parse the vector of strings collected from all child processes
        for result_str in results {
            match result_str.trim().parse::<usize>() {
                Ok(hits) => total_hits += hits,
                Err(_) => Err(SystemError::FailedParsingChildOutput(result_str))?,
            }
        }

        let real_iterations = self.iterations / total_processes * total_processes;
        let result = self.calculate_result(total_hits, real_iterations);

        // Assuming the total iterations were divided equally among processes
        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Processes(total_processes),
        result,
        );
        Ok(())
    }
}
```

factorization.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkMetadata, Benchmarkable};
use crate::task::executor::Executable;
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use std::sync::atomic::{AtomicU64, Ordering};
use std::thread;

pub struct Factorization {
    target_number: u64,
}

impl Default for Factorization {
    fn default() -> Self {
        Self {
            // This is a product of two large primes (4294967291 * 4294967279).
            // It will force the CPU to iterate up to the square root.
            target_number: 18_446_743_979_220_271_189,
        }
    }
}
```

```rust

impl Factorization {
    // Core CPU-bound algorithm for a specific range of potential divisors.
    fn check_range(n: u64, start: u64, end: u64) -> Option<u64> {
        // Force the starting point to be odd if it's greater than 2.
        let mut current = if start.is_multiple_of(2) && start > 2 {
            start + 1
        } else {
            start
        };

        // Handle the only even prime number separately.
        if start <= 2 && 2 <= end {
            if n.is_multiple_of(current) {
                return Some(2);
            }
            current = 3;
        }

        // Check odd numbers in the assigned chunk.
        while current <= end {
            if n.is_multiple_of(current) {
                return Some(current);
            }
            current += 2;
        }

        None
    }

    fn limit(&self) -> u64 {
        // Calculate the upper bound for our search space.
        (self.target_number as f64).sqrt() as u64
    }
}

impl Benchmarkable for Factorization {}
impl Manageable for Factorization {}
impl Measurable for Factorization {}

impl Reportable for Factorization {
    fn name(&self) -> &'static str {
        "CPU-Bound: Factorization"
    }
}

impl Executable for Factorization {
    fn run_sequential(&self) -> Result<(), Error> {
        // let target = self.target_number;
        let target = std::hint::black_box(self.target_number);

        let limit = self.limit();
        let result = Self::check_range(target, 2, limit);

        let result = match result {
            Some(factor) => factor.to_string(),
            None => "None".to_string(),
        };

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Sequential,
```

```
        result
        );

        Ok(())
    }

    fn run_threads(&self, threads: usize) -> Result<(), Error> {
        let limit = self.limit();
        let chunk_size = limit.div_ceil(threads as u64);

        let result = AtomicU64::new(0);

        // Spawn scoped threads to partition the search space.
        thread::scope(|s| {
            for i in 0..threads {
                let target = self.target_number;
                let start = std::cmp::max(2, (i as u64) * chunk_size);
                let end = std::cmp::min(limit, start + chunk_size - 1);

                let result = &result;

                s.spawn(move || {
                    if start <= end
                    && let Some(target) = Self::check_range(target, start, end)
                    {
                        // Attempt to set the result if it's not already set by another thread.
                        result
                        .compare_exchange(
                        0,
                        target,
                        Ordering::SeqCst,
                        Ordering::SeqCst,
                        )
                        .ok();
                    }
                });
            }
        });

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Threads(threads),
        result.load(Ordering::SeqCst)
        );

        Ok(())
    }

    fn run_workers(&self, workers: usize) -> Result<(), Error> {
        let limit = self.limit();

        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::RayonPoolBuild)?;

        // Divide the workload into small chunks for rayon to distribute.
        let chunks_count = workers * 10;
        let chunk_size = limit.div_ceil(chunks_count as u64);

        let target = self.target_number;

```

```
        // Execute the parallel iterator within our custom thread pool.
        let result = pool.install(|| {
            use rayon::prelude::*;

            (0..chunks_count).into_par_iter().find_map_any(|i| {
                let start = std::cmp::max(2, (i as u64) * chunk_size);
                let end = std::cmp::min(limit, start + chunk_size - 1);

                if start <= end {
                    Self::check_range(target, start, end)
                } else {
                    None
                }
            })
        });

        let result = match result {
            Some(factor) => factor.to_string(),
            None => "None".to_string(),
        };

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Workers(workers),
        result
        );

        Ok(())
    }

    fn run_process_chunk(
    &self, process_index: usize, total_processes: usize,
    ) -> Result<(), Error> {
        let limit = self.limit();
        let chunk_size = limit.div_ceil(total_processes as u64);

        let start = std::cmp::max(2, (process_index as u64) * chunk_size);
        let end = std::cmp::min(limit, start + chunk_size - 1);

        let mut found_factor = 0;

        // Execute only the mathematical range assigned to this specific process.
        if start <= end
        && let Some(result) = Self::check_range(self.target_number, start, end)
        {
            found_factor = result;
        }

        // Print the result to stdout so the orchestrator can capture it.
        println!("{}", found_factor);

        Ok(())
    }

    fn aggregate_process_results(
    &self, total_processes: usize, results: Vec<String>,
    ) -> Result<(), Error> {
        let mut final_factor: Option<u64> = None;

        // Parse the vector of strings collected from all child processes.
        for result_str in results {
            match result_str.trim().parse::<u64>() {
```

```
                Ok(factor) => {
                    // If any process found a factor greater than 0, we record it.
                    if factor > 0 {
                        if let Some(existing) = final_factor {
                            // If multiple processes found factors, we take the smallest one.
                            final_factor = Some(existing.min(factor));
                        } else {
                            final_factor = Some(factor);
                        }
                    }
                },
                Err(_) => Err(SystemError::FailedParsingChildOutput(result_str))?,
            }
        }

        // Log the aggregated final result from all processes.
        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Processes(total_processes),
        final_factor
        .map(|factor| factor.to_string())
        .unwrap_or("None".to_string()),
        );

        Ok(())
    }
}
```

primes.rs

```
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkMetadata, Benchmarkable};
use crate::task::executor::Executable;
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use std::ops::RangeInclusive;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

pub struct Primes {
    range: RangeInclusive<u64>,
}

// Builder
impl Primes {
    pub fn with_range(range: RangeInclusive<u64>) -> Self {
        Self { range }
    }
}

impl Primes {
    // Core mathematical algorithm to verify if a single number is prime.
    fn is_prime(n: u64) -> bool {
        if n <= 1 {
            return false;
        }
        if n <= 3 {
            return true;
        }

```

```
        // Eliminate multiples of 2 and 3 early to save CPU cycles.
        if n.is_multiple_of(2) || n.is_multiple_of(3) {
            return false;
        }

        let mut i = 5;
        // Check odd divisors up to the square root of n.
        while i * i <= n {
            if n.is_multiple_of(i) || n.is_multiple_of(i + 2) {
                return false;
            }
            i += 6;
        }

        true
    }
}

impl Benchmarkable for Primes {}
impl Measurable for Primes {}
impl Manageable for Primes {}

impl Reportable for Primes {
    fn name(&self) -> &'static str {
        "CPU-Bound: Primes"
    }
}

impl Executable for Primes {
    fn run_sequential(&self) -> Result<(), Error> {
        let mut total_primes = 0;
        for n in self.range.clone() {
            if Self::is_prime(n) {
                total_primes += 1;
            }
        }

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Sequential,
        total_primes
        );

        Ok(())
    }

    fn run_threads(&self, threads: usize) -> Result<(), Error> {
        let start_bound = *self.range.start();
        let end_bound = *self.range.end();

        // Shared counter to safely accumulate the total primes found by all threads.
        let total_primes = AtomicUsize::new(0);

        thread::scope(|s| {
            for i in 0..threads {
                let shared_total = &total_primes;

                s.spawn(move || {
                    let mut local_count = 0;
                    let mut current = start_bound + i as u64;

                    // Interleaved distribution ensures perfect mathematical load balancing.
```

```
                    while current <= end_bound {
                        if Self::is_prime(current) {
                            local_count += 1;
                        }
                        current += threads as u64;
                    }

                    // Add the local result to the global atomic counter.
                    shared_total.fetch_add(local_count, Ordering::Relaxed);
                });
            }
        });

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Threads(threads),
        total_primes.load(Ordering::SeqCst)
        );

        Ok(())
    }

    fn run_workers(&self, workers: usize) -> Result<(), Error> {
        let start_bound = *self.range.start();
        let end_bound = *self.range.end();

        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::RayonPoolBuild)?;

        let total_primes: usize = pool.install(|| {
            use rayon::prelude::*;

            // Idiomatic Rayon: parallel iterator handles chunks and work-stealing automatically.
            (start_bound..=end_bound)
            .into_par_iter()
            .filter(|&n| Self::is_prime(n))
            .count()
        });

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Workers(workers),
        total_primes
        );

        Ok(())
    }

    fn run_process_chunk(
    &self, process_index: usize, total_processes: usize,
    ) -> Result<(), Error> {
        let start_bound = *self.range.start();
        let end_bound = *self.range.end();

        let mut local_primes = 0;
        let mut current = start_bound + process_index as u64;

        // Execute only the interleaved items assigned to this specific process.
        while current <= end_bound {
```

```rust
            if Self::is_prime(current) {
                local_primes += 1;
            }
            current += total_processes as u64;
        }

        // Print the localized count to stdout for the orchestrator.
        println!("{}", local_primes);

        Ok(())
    }

    fn aggregate_process_results(
    &self, total_processes: usize, results: Vec<String>,
    ) -> Result<(), Error> {
        let mut total_primes = 0;

        // Parse each process output and accumulate the total count.
        for result_str in results {
            match result_str.trim().parse::<usize>() {
                Ok(count) => {
                    total_primes += count;
                },
                Err(_) => Err(SystemError::FailedParsingChildOutput(result_str))?,
            }
        }

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Processes(total_processes),
        total_primes,
        );

        Ok(())
    }
}
```

transpose.rs

```rust
use crate::errors::{Error, SystemError, TaskError};
use crate::task::benchmark::{BenchmarkMetadata, Benchmarkable};
use crate::task::executor::Executable;
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use std::thread;

pub struct Transpose {
    size: usize,
    input_matrix: Vec<u64>,
}

// Builder
impl Transpose {
    pub fn with_size(size: usize) -> Self {
        Transpose {
            size,
            input_matrix: Vec::with_capacity(size * size),
        }
    }
}
```

```rust

impl Transpose {
    pub fn matrix_ref(&self) -> &[u64] {
        &self.input_matrix
    }

    pub fn is_transposed(&self, out_matrix: &[u64]) -> Result<bool, TaskError> {
        // Quick O(1) sanity check to ensure the matrix was transposed correctly.
        // We check off-diagonal elements to verify the swap.

        // Row 0, Col N-1 in the output matrix.
        let index = self.size - 1;
        let top_right_out = out_matrix
        .get(index)
        .ok_or(TaskError::IndexingOutOfBounds(index))?;

        // Row N-1, Col 0 in the input matrix.
        let index = (self.size - 1) * self.size;
        let bottom_left_in = self
        .input_matrix
        .get(index)
        .ok_or(TaskError::IndexingOutOfBounds(index))?;

        // Row 1, Col 2 in the output matrix.
        let index = self.size + 2;
        let arbitrary_out = out_matrix
        .get(index)
        .ok_or(TaskError::IndexingOutOfBounds(index))?;

        // Row 2, Col 1 in the input matrix.
        let index = 2 * self.size + 1;
        let arbitrary_in = self
        .input_matrix
        .get(index)
        .ok_or(TaskError::IndexingOutOfBounds(index))?;

        // Result
        let result = top_right_out == bottom_left_in && arbitrary_out == arbitrary_in;
        Ok(result)
    }
}

impl Benchmarkable for Transpose {}
impl Measurable for Transpose {}

impl Reportable for Transpose {
    fn name(&self) -> &'static str {
        ”Memory-Bound: Matrix Transpose”
    }
}

impl Manageable for Transpose {
    fn setup(&mut self) -> Result<(), Error> {
        let total_elements = self.size * self.size;
        let mut data = vec![0u64; total_elements];

        // Fill the matrix with DETERMINISTIC data based on indices.
        // This guarantees all processes see the exact same virtual matrix.
        for (i, item) in data.iter_mut().enumerate() {
            *item = (i % 500) as u64;
        }

        self.input_matrix = data;
```

```rust

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        // Clear the input matrix to free memory.
        self.input_matrix.clear();
        Ok(())
    }
}

impl Executable for Transpose {
    fn run_sequential(&self) -> Result<(), Error> {
        let n = self.size;
        let input = self.matrix_ref();
        let mut out = vec![0u64; n * n];

        // Standard sequential transposition.
        for r in 0..n {
            for c in 0..n {
                let out_index = r * n + c;
                let out = out
                .get_mut(out_index)
                .ok_or(TaskError::IndexingOutOfBounds(out_index))?;

                let in_index = c * n + r;
                let in_value = input
                .get(in_index)
                .ok_or(TaskError::IndexingOutOfBounds(in_index))?;

                *out = *in_value;
            }
        }

        // Ensuring the logic is correct and preventing false positives in benchmarks.
        let result = self.is_transposed(&out)?;

        log::info!(
        "TASK: {}, MODE: {}, IS TRANSPOSED: {}",
        self.name(),
        BenchmarkMetadata::Sequential,
        result
        );

        Ok(())
    }

    fn run_threads(&self, threads: usize) -> Result<(), Error> {
        let n = self.size;
        let input = self.matrix_ref();
        let mut out = vec![0u64; n * n];

        let chunk_rows = n.div_ceil(threads);
        let chunk_size_elements = chunk_rows * n;

        thread::scope(|s| {
            for (i, chunk) in out.chunks_mut(chunk_size_elements).enumerate() {
                s.spawn(move || {
                    let start_row = i * chunk_rows;
                    let local_rows = chunk.len() / n;

                    for r_local in 0..local_rows {
                        let r_global = start_row + r_local;
```

```
                        for c in 0..n {
                            let out_index = r_local * n + c;
                            let in_index = c * n + r_global;

                            let out = chunk
                            .get_mut(out_index)
                            .ok_or(TaskError::IndexingOutOfBounds(out_index));

                            let in_value = input
                            .get(in_index)
                            .ok_or(TaskError::IndexingOutOfBounds(in_index));

                            match (out, in_value) {
                                (Ok(out), Ok(in_value)) => {
                                    *out = *in_value;
                                },
                                (Err(e), _) | (_, Err(e)) => {
                                    log::error!(
                                    "Thread processing row {}: Error - {}",
                                    r_global,
                                    e
                                    );
                                    std::process::exit(1);
                                },
                            }
                        }
                    }
                });
            }
        });

        let result = self.is_transposed(&out)?;

        log::info!(
        "TASK: {}, MODE: {}, IS TRANSPOSED: {}",
        self.name(),
        BenchmarkMetadata::Threads(threads),
        result
        );

        Ok(())
    }

    fn run_workers(&self, workers: usize) -> Result<(), Error> {
        let n = self.size;
        let input = self.matrix_ref();
        let mut out = vec![0u64; n * n];

        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::RayonPoolBuild)?;

        pool.install(|| {
            use rayon::prelude::*;

            // Process each row in parallel using Rayon's iterators over mutable chunks.
            out.par_chunks_mut(n)
            .enumerate()
            .for_each(|(r, row_slice)| {
                for c in 0..n {
                    let in_index = c * n + r;
                    let in_value = input
```

```
                    .get(in_index)
                    .ok_or(TaskError::IndexingOutOfBounds(in_index));

                    let out = row_slice
                    .get_mut(c)
                    .ok_or(TaskError::IndexingOutOfBounds(c));

                    match (out, in_value) {
                        (Ok(out), Ok(in_value)) => {
                            *out = *in_value;
                        },
                        (Err(e), _) | (_, Err(e)) => {
                            log::error!("Worker processing row {}: Error - {}", r, e);
                            std::process::exit(1);
                        },
                    }
                }
            });
        });

        // Ensuring the logic is correct and preventing false positives in benchmarks.
        let result = self.is_transposed(&out)?;

        log::info!(
        "TASK: {}, MODE: {}, IS TRANSPOSED: {}",
        self.name(),
        BenchmarkMetadata::Workers(workers),
        result
        );

        Ok(())
    }

    fn run_process_chunk(
    &self, process_index: usize, total_processes: usize,
    ) -> Result<(), Error> {
        let n = self.size;

        // Handle process memory isolation deterministically.
        let mut fallback_input;
        let input = if self.input_matrix.is_empty() {
            let total_elements = n * n;
            fallback_input = vec![0u64; total_elements];
            for (i, item) in fallback_input.iter_mut().enumerate() {
                *item = (i % 500) as u64;
            }
            &fallback_input
        } else {
            self.matrix_ref()
        };

        let chunk_rows = n.div_ceil(total_processes);
        let start_row = process_index * chunk_rows;
        let end_row = std::cmp::min(n, start_row + chunk_rows);

        let mut control_value = 0;

        if start_row < end_row {
            let mut out_chunk = vec![0u64; (end_row - start_row) * n];

            for r_local in 0..(end_row - start_row) {
                let r_global = start_row + r_local;
                for c in 0..n {
```

```
                    let in_index = c * n + r_global;
                    let out_index = r_local * n + c;

                    let out = out_chunk
                    .get_mut(out_index)
                    .ok_or(TaskError::IndexingOutOfBounds(out_index))?;

                    let in_value = input
                    .get(in_index)
                    .ok_or(TaskError::IndexingOutOfBounds(in_index))?;

                    *out = *in_value;
                }
            }

            std::hint::black_box(&mut out_chunk);
            control_value = out_chunk.first().copied().unwrap_or(0);
        }

        println!("{}", control_value);

        Ok(())
    }

    fn aggregate_process_results(
    &self, total_processes: usize, results: Vec<String>,
    ) -> Result<(), Error> {
        let mut final_control_value = 0;

        for result_str in results {
            match result_str.trim().parse::<u32>() {
                Ok(value) => {
                    // Capture any valid control value to confirm execution.
                    final_control_value = value;
                },
                Err(_) => Err(SystemError::FailedParsingChildOutput(result_str))?,
            }
        }

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Processes(total_processes),
        final_control_value,
        );

        Ok(())
    }
}
```

io.rs

```
use crate::errors::{Error, SetupError, SystemError, TaskError, TeardownError};
use crate::task::benchmark::{BenchmarkMetadata, Benchmarkable};
use crate::task::executor::Executable;
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use std::fs::{self, File};
use std::io::{Read, Write};
use std::path::{Path, PathBuf};
use std::sync::atomic::{AtomicUsize, Ordering};
```

```rust
use std::thread;

pub struct WordCount {
    base_directory: PathBuf,
    files_amount: usize,
    words_per_file: usize,
}

impl WordCount {
    pub fn new(files_amount: usize, words_per_file: usize) -> Self {
        let mut base_directory = std::env::temp_dir();
        base_directory.push("MTTP-WordCount-Temp");

        Self {
            base_directory,
            files_amount,
            words_per_file,
        }
    }

    // Helper to recursively collect all file paths to process.
    // Sorting ensures deterministic chunking for parallel processes.
    fn collect_files(&self) -> Result<Vec<PathBuf>, Error> {
        let mut files = self.visit_dirs(&self.base_directory)?;
        files.sort();
        Ok(files)
    }

    fn visit_dirs(&self, directory: &Path) -> Result<Vec<PathBuf>, SystemError> {
        if !directory.is_dir() {
            Err(SystemError::DirectoryExpected(
            directory.display().to_string(),
            ))?;
        }

        let directory_entries =
        fs::read_dir(directory).map_err(SystemError::DirectoryRead)?;

        let mut paths = vec![];
        for entry in directory_entries {
            let entry = entry.map_err(SystemError::DirectoryEntry)?;
            let path = entry.path();
            if path.is_dir() {
                let inner_files = self.visit_dirs(&path)?;
                paths.extend(inner_files);
            } else {
                paths.push(path);
            }
        }

        Ok(paths)
    }

    // Core operation: Read a file from disk and count its words.
    fn count_words(path: &Path) -> Result<usize, Error> {
        let mut file = File::open(path).map_err(SystemError::FileOpen)?;

        let mut contents = String::new();
        file.read_to_string(&mut contents)
        .map_err(SystemError::FileRead)?;

        Ok(contents.split_whitespace().count())
    }
```

```rust
}

impl Benchmarkable for WordCount {}
impl Measurable for WordCount {}

impl Reportable for WordCount {
    fn name(&self) -> &'static str {
        "I/O-Bound: Word Count"
    }
}

impl Manageable for WordCount {
    fn setup(&mut self) -> Result<(), Error> {
        // Clean up any previous runs just in case.
        let _ = self.teardown();

        fs::create_dir_all(&self.base_directory)
        .map_err(SetupError::TemporaryDirectoryCreate)?;

        let text_content = "benchmark ".repeat(self.words_per_file);

        // Create deep directory structures to force recursive searching.
        for i in 0..self.files_amount {
            // Group every 100 files into a subdirectory.
            let sub_directory = self.base_directory.join(format!("folder_{}", i / 100));
            fs::create_dir_all(&sub_directory).map_err(SetupError::CreateFolder)?;

            let file_path = sub_directory.join(format!("file_{}.txt", i));
            let mut file = File::create(&file_path).map_err(SetupError::FileCreation)?;

            file.write_all(text_content.as_bytes())
            .map_err(SetupError::FileWrite)?;
        }

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        // Remove the temporary directory and all generated files.
        fs::remove_dir_all(&self.base_directory)
        .map_err(TeardownError::TemporaryDirectoryDelete)?;
        Ok(())
    }
}

impl Executable for WordCount {
    fn run_sequential(&self) -> Result<(), Error> {
        let files = self.collect_files()?;
        let mut total_words = 0;

        for path in files {
            total_words += Self::count_words(&path)?;
        }

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Sequential,
        total_words
        );

        Ok(())
    }
```

```rust

    fn run_threads(&self, threads: usize) -> Result<(), Error> {
        if threads == 0 {
            return Ok(());
        }

        let files = self.collect_files()?;
        let chunk_size = files.len().div_ceil(threads);

        // Use an atomic to safely aggregate the total word count.
        let total_words = AtomicUsize::new(0);

        thread::scope(|s| {
            for chunk in files.chunks(chunk_size) {
                let shared_total = &total_words;
                s.spawn(move || {
                    let mut local_count = 0;
                    for path in chunk {
                        match Self::count_words(path) {
                            Ok(amount) => local_count += amount,
                            Err(err) => {
                                log::error!(
                                "Error processing file {}: {}",
                                path.display(),
                                err
                                );
                                std::process::exit(1);
                            },
                        }
                    }
                    shared_total.fetch_add(local_count, Ordering::Relaxed);
                });
            }
        });

        log::info!(
        "TASK: {}, MODE: {}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Threads(threads),
        total_words.load(std::sync::atomic::Ordering::Relaxed)
        );

        Ok(())
    }

    fn run_workers(&self, workers: usize) -> Result<(), Error> {
        if workers == 0 {
            return Ok(());
        }

        let files = self.collect_files()?;

        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::RayonPoolBuild)?;

        let total_words: usize = pool.install(|| {
            use rayon::prelude::*;

            // Rayon dynamically handles the work stealing across I/O requests.
            files
            .into_par_iter()
```

```
            .map(|path| match Self::count_words(&path) {
                Ok(amount) => amount,
                Err(err) => {
                    log::error!("Error processing file {}: {}", path.display(), err);
                    0
                },
            })
            .sum()
        });

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Workers(workers),
        total_words
        );

        Ok(())
    }

    fn run_process_chunk(
    &self, process_index: usize, total_processes: usize,
    ) -> Result<(), Error> {
        if total_processes == 0 {
            return Ok(());
        }

        let files = self.collect_files()?;

        let chunk_size = files.len().div_ceil(total_processes);
        let start_idx = process_index * chunk_size;
        let end_idx = std::cmp::min(files.len(), start_idx + chunk_size);

        let mut local_words = 0;

        if start_idx < end_idx {
            let chunk = files
            .get(start_idx..end_idx)
            .ok_or(TaskError::ChunkOutOfBounds(start_idx, end_idx))?;

            for path in chunk {
                match Self::count_words(path) {
                    Ok(count) => local_words += count,
                    Err(err) => {
                        log::error!("Error processing file {}: {}", path.display(), err);
                        std::process::exit(1);
                    },
                }
            }
        }

        // Print to stdout for the orchestrator to capture.
        println!("{}", local_words);

        Ok(())
    }

    fn aggregate_process_results(
    &self, total_processes: usize, results: Vec<String>,
    ) -> Result<(), Error> {
        let mut total_words = 0;

        for result_str in results {
```

```
            match result_str.trim().parse::<usize>() {
                Ok(count) => {
                    total_words += count;
                },
                Err(_) => Err(SystemError::FailedParsingChildOutput(result_str))?,
            }
        }

        log::info!(
        "TASK: {}, MODE: {:?}, RESULT: {}",
        self.name(),
        BenchmarkMetadata::Processes(total_processes),
        total_words,
        );

        Ok(())
    }
}
```